

THE SCENARIO GENERATION ALGORITHM FOR MULTISTAGE STOCHASTIC LINEAR PROGRAMMING

Michael S. Casey

Department of Mathematics and Computer Science, University of Puget Sound, Tacoma, Washington
email: mcasey@ups.edu <http://www.math.ups.edu/~mcasey/>

Suvrajeet Sen

SIE Department, University of Arizona, Tucson, Arizona
email: sen@sie.arizona.edu

A multistage stochastic linear program (MSLP) is a model of sequential stochastic optimization where the objective and constraints are linear. When any of the random variables used in the MSLP are continuous, the problem is infinite dimensional. In order to numerically tackle such a problem we usually replace it with a finite dimensional approximation. Even when all the random variables have finite support, the problem is often computationally intractable and must be approximated by a problem of smaller dimension. One of the primary challenges in the field of stochastic programming deals with discovering effective ways to evaluate the importance of scenarios, and to use that information to trim the scenario tree in such a way that the solution to the smaller optimization problem is not much different than the problem stated with the original tree. The Scenario Generation (SG) algorithm proposed in this paper is a finite element method that addresses this problem for the class of MSLP with random right-hand sides.

Key words: multistage stochastic linear program; stochastic optimization; scenario tree generation; finite element methods

MSC2000 Subject Classification: Primary: 90C15, 90C25, 90C06

OR/MS subject classification: Primary: Programming/stochastic

1. Introduction. Multistage sequential decision problems arise in a variety of applications: finance (Carino et al. [2]), production systems (Boskma et al. [14]), power generation (Nowak and Romisch [10]) and many more. The inherent uncertainty of data (e.g. costs, prices, demands, availability), together with the sequential evolution of data over time, leads to a sequential optimization-under-uncertainty model. Typically such models take one of two forms: (i) multistage stochastic programs (MSP) or (ii) stochastic dynamic programs (e.g. Markov decision processes). While stochastic dynamic programming (SDP) may be an appropriate approach for certain situations, many realistic applications call for a far larger number of state variables than SDP can accommodate efficiently. For large scale realistic applications with many state variables and constraints, MSP provides an appropriate modeling tool. However, the current state-of-the-art for MSP has its own computational limitations. One of the prerequisites for an MSP model is a discretization of the stochastic process representing the evolution of the random data. Although the random variables comprising this process may be continuous, the MSP typically must be solved numerically and this requires discrete random variables (in fact they must have only a finite number of outcomes); this discrete process can be represented by a *scenario tree* and it is usually a *discretization* of a continuous process or an *aggregation* of a discrete process.

As one might expect, there is both a science and an art to such discretization/aggregation of the data evolution process. The science arises in the approximation of the data evolution process; the art comes from the realization that fine discretizations lead to numerically impossible problems, whereas, coarse discretizations may lead to a questionable model by overlooking important events. The right balance is not easy to strike. As for the scientific approach to scenario generation, we can identify two related approaches. One is based on statistical approximations (e.g. moment/target matching as in Hoyland and Wallace [8]), and the other on approximation theory (as in Pflug [12] and Growe-Kuska et al. [9]). As for the art of scenario generation, there is growing folklore in the stochastic programming community that the scenario tree needs to account for *low probability, high cost* (sometimes referred to as catastrophic) events. Unfortunately, such qualitative guidelines are difficult to quantify and implement.

In practice, the true MSP (say P) may be far too large to solve using even the best of algorithms, on the fastest of computers. Hence, the approach adopted in the stochastic programming literature replaces the problem P by another problem $\hat{P}$ which is formulated by replacing the true stochastic process underlying P with a coarse-grain approximation. The point of view adopted in this paper is based on approximating the true (fine-grain) MSP denoted P by a sequence of approximations P_k which

ultimately lead to a solution of P . To generate these smaller problems P_k , we approximate the decision problem rather than focus exclusively on approximating the stochastic process underlying the decision problem. This viewpoint has already been adopted for two-stage stochastic linear programming problems (e.g. Frauendorfer and Kall [7], Edirisinghe and Ziemba [5] among others). However, extensions of this idea to multistage stochastic linear programming (MSLP) have had mixed success. Frauendorfer [6] proposes the development of upper and lower bounding trees based on barycentric approximations. While this approach leads to approximations that are asymptotically convergent, the upper bounding scheme requires the solution of multiple MSLPs in each iteration, thus incurring fairly extensive computations in each iteration. To the best of our knowledge, there has been one other attempt at designing a successive refinement algorithm for MSLP: Edirisinghe [4]. This algorithm is based on forming aggregations of the nonanticipativity constraints. While this method does not incur some of the computational overheads of the method proposed in Frauendorfer [6], it is not clear that the method provides any asymptotic guarantees.

In this paper, we study an algorithm for solving MSLP which uses some of the same tools as in two-stage SLP, thus keeping the computations manageable. At the same time, this approach also provides asymptotic optimality and, as we will demonstrate subsequently, is able to take advantage of parallel/distributed computing capability. Among some of its other contributions, we should mention that our algorithm provides an operational realization of the notion of *prolongations* first introduced in Olsen [11]. Olsen's use of prolongations was motivated by the mathematics of his convergence result. In this paper, our prolongation not only provides the basis for our convergence result, but also provides operating policies for extending the primal solution of an approximation to a solution of the original problem. While such a prolongation may not yield a feasible solution for all possible scenarios, our method provides a probability of satisfying feasibility. This is an important step for implementing solutions of MSLP obtained from approximating the original problem P .

2. Preliminaries. Stochastic linear programming is one of the more powerful paradigms for decision making under uncertainty. Mathematically, we have a time index $t \in \{1, \dots, T\}$ and a time horizon consisting of T stages. Uncertainty is modeled by a filtered probability space $(\Xi, \mathcal{S}, \{\mathcal{F}_t\}_{t=1}^T, P)$. The sample space Ξ is defined as $\Xi := \Xi^1 \times \dots \times \Xi^T$ where $\Xi^t \subset \mathbb{R}^{r_t}$ with r_t a positive integer. An outcome is $\vec{\xi} := (\xi_1, \dots, \xi_T) \in \Xi$ and $\vec{\xi}_t := (\xi_1, \dots, \xi_t)$. The σ -algebra $\mathcal{S}$ is the set of events that are assigned probabilities by the probability measure P and $\{\mathcal{F}_t\}_{t=1}^T$ is a filtration on $\mathcal{S}$. The (recourse) decision at stage t is the random variable $x_t : \Xi \rightarrow \mathbb{R}^{n_t}$ where n_t is a positive integer. The cost of the decision at time t is the random variable $c_t : \Xi \rightarrow \mathbb{R}^{n_t}$. For all $t \in \{1, \dots, T\}$ and all $\tau \in \{1, \dots, T\}$, $b_t : \Xi \rightarrow \mathbb{R}^{m_t}$ and $A_{t\tau}$ is an $m_t \times n_t$ real-valued matrix. For a fuller discussion of the above probability terms we refer the reader to Durrett [3].

One formulation of an MSLP is as follows:

$$\begin{aligned}
 \min_{x_1, \dots, x_T} \quad & c_1^\top x_1 + E \sum_{t=2}^T c_t^\top(\vec{\xi}_t) x_t(\vec{\xi}_t) \\
 \text{subject to} \quad & A_{11} x_1 = b_1 \\
 & A_{t1} x_1 + \sum_{\tau=2}^t A_{t\tau} x_\tau(\vec{\xi}_\tau) = b_t(\vec{\xi}_t) \\
 & x_1 \geq 0, x_t(\vec{\xi}_t) \geq 0 \\
 & x_t \in L^2(\Xi^1 \times \dots \times \Xi^t, \mathbb{R}^{n_t}) \quad \text{a.s., } t = 2, \dots, T
 \end{aligned}$$

In the above formulation the constraints hold almost surely (a.s.). When we say that a policy $x = (x_1, \dots, x_T)$ is adapted to the filtration $\mathcal{F} = \{\mathcal{F}_t\}_{t=1}^T$, denoted $x \in \mathcal{F}$, we mean that each x_t is measurable with respect to the corresponding σ -algebra $\mathcal{F}_t$, i.e. $x_t \in \mathcal{F}_t$. Such a policy is also said to be *nonanticipative* since x_t is a function of $(\xi_1, \dots, \xi_t)$ but not of the random vector $(\xi_{t+1}, \dots, \xi_T)$. We can reformulate the MSLP in such a way that the role played by $\{\mathcal{F}_t\}_{t=1}^T$ is brought to the fore:

$$\begin{aligned} \min \quad & E \sum_{t=1}^T E[c_t^\top | \mathcal{F}_t] x_t \\ \text{subject to} \quad & \begin{cases} E[\sum_{\tau=1}^t A_{t\tau} x_t | \mathcal{F}_t] = E[b_t | \mathcal{F}_t] & \text{a.s for } t = 1, \dots, T \\ x_t \geq 0 \\ x = (x_1, \dots, x_T) \in \mathcal{F} \\ x_t \in L^2(\Xi, \mathbb{R}^{n_t}) \end{cases} \end{aligned} \quad (1)$$

As in Wright [16] we refer to this problem as $P(\mathcal{F}, \mathcal{F})$. The problem $P(\mathcal{F}^d, \mathcal{F}^c)$ uses two filtrations: the first argument ($\mathcal{F}^d$) denotes the filtration with respect to which the decisions are adapted, whereas the second argument ($\mathcal{F}^c$) denotes the filtration with respect to which the equality constraints are adapted.

The decision policy x is adapted to $\mathcal{F}^d$ if $x \in \mathcal{F}^d$. The equality constraint $\sum_{\tau=1}^t A_{t\tau} x_t = b_t$ is adapted to the filtration $\mathcal{F}^c$ by replacing both sides of the equality with conditional expectation with respect to $\mathcal{F}_t^c$, i.e. $E[\sum_{\tau=1}^t A_{t\tau} x_t | \mathcal{F}_t^c] = E[b_t | \mathcal{F}_t^c]$. We say that the filtration $\mathcal{G} = \{\mathcal{G}_t\}_{t=1}^T$ is coarser than the filtration $\mathcal{F} = \{\mathcal{F}_t\}_{t=1}^T$ if $\mathcal{G}_t \subset \mathcal{F}_t$ for each $t \in \{1, \dots, T\}$.

Let $\hat{\mathcal{F}}$ be coarser than $\mathcal{F}$. The decision aggregated problem $P(\hat{\mathcal{F}}, \mathcal{F})$ is

$$\begin{aligned} \min \quad & E \sum_{t=1}^T E[c_t^\top | \hat{\mathcal{F}}_t] x_t \\ \text{subject to} \quad & \begin{cases} E[\sum_{\tau=1}^t A_{t\tau} x_t | \mathcal{F}_t] = E[b_t | \mathcal{F}_t] & \text{a.s for } t = 1, \dots, T \\ x_t \geq 0 \\ x = (x_1, \dots, x_T) \in \hat{\mathcal{F}} \\ x_t \in L^2(\Xi, \mathbb{R}^{n_t}) \end{cases} \end{aligned}$$

We can also form the *constraint* aggregated problem $P(\mathcal{F}, \hat{\mathcal{F}})$:

$$\begin{aligned} \min \quad & E \sum_{t=1}^T E[c_t^\top | \mathcal{F}_t] x_t \\ \text{subject to} \quad & \begin{cases} E[\sum_{\tau=1}^t A_{t\tau} x_t | \hat{\mathcal{F}}_t] = E[b_t | \hat{\mathcal{F}}_t] & \text{a.s for } t = 1, \dots, T \\ x_t \geq 0 \\ x = (x_1, \dots, x_T) \in \mathcal{F} \\ x_t \in L^2(\Xi, \mathbb{R}^{n_t}) \end{cases} \end{aligned}$$

The *fully* aggregated problem $P(\hat{\mathcal{F}}, \hat{\mathcal{F}})$ is

$$\begin{aligned} \min \quad & E \sum_{t=1}^T E[c_t^\top \mid \hat{\mathcal{F}}_t] x_t \\ \text{subject to} \quad & \begin{cases} E[\sum_{\tau=1}^t A_{t\tau} x_\tau \mid \hat{\mathcal{F}}_t] = E[b_t \mid \hat{\mathcal{F}}_t] & \text{a.s for } t = 1, \dots, T \\ x_t \geq 0 \\ x = (x_1, \dots, x_T) \in \hat{\mathcal{F}} \\ x_t \in L^2(\Xi, \mathbb{R}^{n_t}) \end{cases} \end{aligned}$$

Of course, if $\mathcal{F}^1$ and $\mathcal{F}^2$ are two filtrations with $\mathcal{F}^1$ coarser than $\mathcal{F}^2$, then the constraints in $P(\mathcal{F}, \mathcal{F}^2)$ are more restrictive than those in $P(\mathcal{F}, \mathcal{F}^1)$. Hence, denoting the value of $P(\mathcal{F}, \mathcal{F}^1)$ and $P(\mathcal{F}, \mathcal{F}^2)$ by $v(P(\mathcal{F}, \mathcal{F}^1))$ and $v(P(\mathcal{F}, \mathcal{F}^2))$ respectively, we have

$$v(P(\mathcal{F}, \mathcal{F}^1)) \leq v(P(\mathcal{F}, \mathcal{F}^2)). \quad (2)$$

Now consider an MSLP where only b_t is random, i.e. the stochastic program has only random right-hand sides with all other data deterministic. Then the value of the fully aggregated problem $P(\hat{\mathcal{F}}, \hat{\mathcal{F}})$ is the same as the value of the constraint aggregated problem $P(\mathcal{F}, \hat{\mathcal{F}})$ (Wright [16, Theorem 9]). This implies that if we consider optimizing a feasible MSLP with random right-hand sides over a sequence of filtrations $\{\mathcal{F}^k\}_{k=1}^\infty$ with $\mathcal{F}_t^k \subseteq \mathcal{F}_t^{k+1}$ for all $t \in \{1, \dots, T\}$ and we let the optimal value of $P(\mathcal{F}^k, \mathcal{F}^k)$ be denoted v^k , then v^k is a monotonically increasing sequence of real numbers bounded above by $v(P(\mathcal{F}, \mathcal{F}))$. Since this provides the motivation for the sequence of approximations generated by our algorithm, we formalize the result in the following theorem.

Theorem 2.1. *Assume that the true problem $P(\mathcal{F}, \mathcal{F})$ is feasible. Let $\{\mathcal{F}^k\}_{k=1}^\infty$ be a sequence of filtrations such that $\mathcal{F}^k$ is coarser than $\mathcal{F}^{k+1}$. Let v^k be the optimal value of $P(\mathcal{F}^k, \mathcal{F}^k)$. Then $\{v^k\}_{k=1}^\infty$ is a monotonically increasing sequence of real numbers.*

3. Scenario Trees and Filtrations. When all the random variables have finite support, the stochasticity present in a multistage model can be represented by a scenario tree. Since algorithms work with discretizations, in this section we make the connection between scenario trees (which are discrete) and filtrations which work for both continuous as well as discrete random variables. Our discussion of scenario trees follows Rockafeller [15, pg 2-3].

A *scenario tree* $\mathcal{T} = (\mathcal{N}, \mathcal{A})$ is a rooted tree where all the leaves are at depth T . The set of nodes at depth t is denoted $\mathcal{N}^t$ and thus the set of nodes is $\mathcal{N} = \bigcup_t \mathcal{N}^t$. The set of *children* of node n is denoted by C_n . Each arc (n, m) has an associated conditional probability q_{nm} of transition to m given that n is reached. Thus $\sum_{m \in C_n} q_{nm} = 1$.

Alternatively, the scenario tree can be described using a filtration where σ -algebras represent information available to the decision maker. Assuming ξ_t has finite support, the σ -algebra $\mathcal{F}_t = \sigma(\xi_t)$ generated by ξ_t is finite and so is the filtration $\{\mathcal{F}_1, \dots, \mathcal{F}_T\}$. Since $\mathcal{F}_t$ is finite it is generated by a finite partition B_j^t of Ξ :

$$\Xi = \bigcup_{i=1}^{k_t} B_i^t \quad \text{with} \quad B_j^t \cap B_l^t = \emptyset \quad \text{for } j \neq l. \quad (3)$$

The same is true for $\mathcal{F}_{t+1}$:

$$\Xi = \bigcup_{i=1}^{k_{t+1}} B_i^{t+1} \quad \text{with} \quad B_j^{t+1} \cap B_l^{t+1} = \emptyset \quad \text{for } j \neq l. \quad (4)$$

The filtration property implies a relationship between the two partitions B_j^t and B_j^{t+1} :

$$\forall i, 1 \leq i \leq k_t, \exists J_i^{t+1} \subseteq \{1, \dots, k_{t+1}\} \text{ with } B_i^t = \bigcup_{k \in J_i^{t+1}} B_k^{t+1}. \quad (5)$$

Recall that when we say a policy $x = \{x_t(\vec{\xi}), t = 1, \dots, T\}$ is adapted to the filtration $\{\mathcal{F}_t\}_{t=1}^T$ we mean that each x_t is measurable with respect to the corresponding σ -algebra $\mathcal{F}_t$. This implies

$$x \text{ is } \mathcal{F}\text{-adapted} \Leftrightarrow x_t(\vec{\xi}) = \text{const} \quad \forall \vec{\xi} \in B_i^t, \forall i, t.$$

The connection between the filtration $\mathcal{F}$ and the scenario tree can be made explicit by identifying the components B_i^t of the partitions with the corresponding nodes of the scenario tree.

We use the following notation for discussing scenario trees:

- $\mathcal{T} = (\mathcal{N}, \mathcal{A})$, the scenario tree is a rooted tree where each node n belongs to stage t_n , i.e. $n \in \mathcal{N}^{t_n}$. We have
 - $n = 1$ with $t_1 = 1$ is the one and only root,
 - the nodes $\{n | t_n = T\}$ are the leaves, and
 - the unique path from the root $n = 1$ to any leaf n with $t_n = T$ is a scenario;
- S is the set of scenarios (root-to-leaf paths);
- p_s is the probability of scenario $s \in S$;
- S_n is the set of scenarios passing through node n ;
- $\bar{p}_n = \sum_{s \in S_n} p_s$ is the probability of ever reaching node n ;
- $H_n \subseteq \mathcal{N}$ are the predecessor nodes or *history* of node n .
- h_n is the immediate predecessor or *parent node* of node $n \in \mathcal{N}, n \geq 2$;
- $F_{n,s} \subseteq \mathcal{N}$ are the successor nodes or the *future of n on scenario s* ;
- $F_n = \bigcup_{s \in S_n} F_{n,s}$ are the successor nodes of n or the *future of $n \in \mathcal{N}$* ;
- $C_n \subseteq \mathcal{N}$ are the immediate successor nodes of n or the *children of n* ;
- $q_{n,m}$ is the conditional probability of reaching node m given that node n has been reached;
- $\bar{B}_n \subseteq \Xi$ is the set of scenarios represented by node n ; and finally,
- $c_n \equiv c_{t_n}(\xi)$ and $A_{nm} \equiv A_{t_n t_m}(\xi)$.

Let $\hat{\mathcal{F}}$ be the filtration corresponding to the tree $\mathcal{T}$. The MSLP over the scenario tree $\mathcal{T}$ is the fully aggregated problem $P(\hat{\mathcal{F}}, \hat{\mathcal{F}})$. This fully aggregated problem is a linear program and can be written as

$$\begin{aligned} \min \quad & \sum_{n \in \mathcal{N}: \bar{p}_n > 0} \bar{p}_n c_n^\top x_n \\ & \sum_{m \in H_n: \bar{p}_m > 0} A_{nm} x_m + A_{nn} x_n = b_n \\ & x_n \geq 0 \quad \forall n: \bar{p}_n > 0 \end{aligned} \quad (6)$$

The dual to this problem is

$$\begin{aligned} \max \quad & \sum_{n \in \mathcal{N}: \bar{p}_n > 0} b_n^\top u_n \\ & A_{nn}^\top u_n + \sum_{m \in F_n: \bar{p}_m > 0} A_{mn}^\top u_m \leq \bar{p}_n c_n \quad \forall n: \bar{p}_n > 0 \end{aligned} \quad (7)$$

By replacing the dual variables u_n by $\bar{p}_n u_n$ and dividing the n -th constraint by $\bar{p}_n$, we obtain the following dual problem

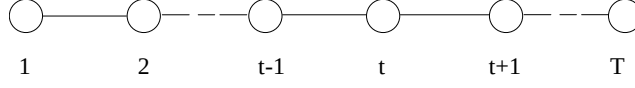


Figure 1: Degenerate tree

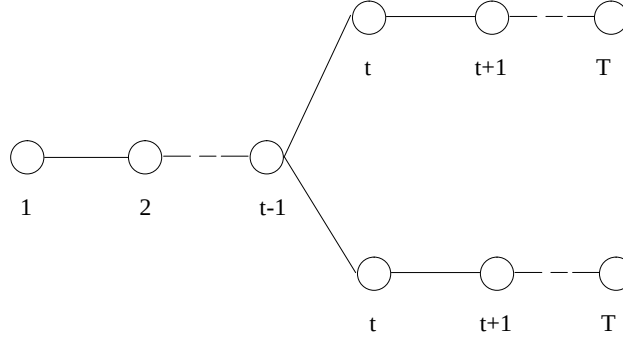


Figure 2: First partition

$$\begin{aligned} \max \quad & \sum_{n \in \mathcal{N}: \bar{p}_n > 0} \bar{p}_n b_n^\top \pi_n \\ & A_{nn}^\top \pi_n + \sum_{m \in F_n: \bar{p}_m > 0} q_{n,m} A_{mn}^\top \pi_m \leq c_n \quad \forall n: \bar{p}_n > 0 \end{aligned} \quad (8)$$

We will denote the optimal solutions to these problems as $\bar{x}_n$ and $\bar{\pi}_n$ respectively. The scaled version of the dual provided above has an interesting probabilistic interpretation. In this form, the dual vector $\bar{\pi}_n$ represents the conditional marginal value of resources (conditioned on arrival at node n). The dual feasibility constraints are reminiscent of dynamic programming optimality conditions requiring that the marginal value of resources at node n plus the salvage value for the future, must not exceed c_n , the cost at node n .

Degenerate Subfiltration

There is a one-to-one correspondence between finite filtrations and scenario trees. We present in this subsection the degenerate (sub)filtration and its associated degenerate tree. We show what happens to this filtration when we split a node of this tree. We can define the *degenerate subfiltration* $\hat{\mathcal{F}}$ by saying that $\hat{\mathcal{F}}_t = \{\Xi, \emptyset\}$. The corresponding scenario tree is called the *degenerate tree*; it consists of only one scenario (Figure 1). If we partition a particular Ξ^t into two subsets Ξ_1^t and Ξ_2^t then the new tree can be depicted as in Figure 2. The two nodes in stage t represent the two subsets. The new filtration is denoted $\hat{\hat{\mathcal{F}}}$. This filtration is such that $\hat{\hat{\mathcal{F}}}_\tau = \hat{\mathcal{F}}_\tau$ for $\tau < t$ and $\hat{\hat{\mathcal{F}}}_s = \sigma(\{\Xi^1 \times \dots \times \Xi_1^t \times \dots \times \Xi^T, \Xi^1 \times \dots \times \Xi_2^t \times \dots \times \Xi^T, \emptyset\})$ for $s \geq t$.

Tree Update Procedure

The *tree update* procedure described here provides a systematic method for creating a sequence of subfiltrations, each of which is finer than its predecessor. Given a tree $\mathcal{T}$ this method will provide a new scenario tree $\mathcal{T}^+$. The procedure assumes that a node n has been identified (see section 4) and the sample paths assigned to this node will be partitioned into two subsets to yield a finer discretization. The procedure consists of four steps:

- **Step 1** Let $m \in \mathcal{N}$ be such that $m \notin \mathcal{N}$. Note that m is the new node.

- **Step 2** Let $\mathcal{T}_n = (\mathcal{A}_n, \mathcal{N}_n)$ be the rooted subtree of $\mathcal{T}$ with root node n .
- **Step 3** Let $\mathcal{T}_m = (\mathcal{A}_m, \mathcal{N}_m)$ denote a tree that is isomorphic to the subtree $\mathcal{T}_n$; that is, the tree $\mathcal{T}_m$ has the same graph properties as tree $\mathcal{T}_n$. Let the root node of $\mathcal{T}_m$ be m .
- **Step 4** Let $\mathcal{T}^+ = (\mathcal{N}^+, \mathcal{A}^+)$ with
 - $\mathcal{N}^+ = \mathcal{N} \cup \mathcal{N}_m$ and
 - $\mathcal{A}^+ = \mathcal{A} \cup \mathcal{A}_m \cup \{(h_n, m)\}$.

For each $n \in \mathcal{N}^+$ calculate $\bar{p}_n$.

4. The Scenario Generation Algorithm. We restrict ourselves to MSLP which have random right-hand sides $b(\vec{\xi})$ only; all other data are deterministic. In addition, we require b_t to be an affine function of $\vec{\xi}$. The initial tree may be the degenerate one (obtained by replacing the random variables by their expectations), or any other convenient approximation. Given such an initial tree, we formulate over it the fully aggregated problem which is finite-dimensional and can be viewed as an LP. This LP can be solved using either specialized algorithms or, in some cases standard LP solvers.

A solution to this fully aggregated problem produces at each node n (i) a primal decision $\bar{x}_n$ and (ii) a dual multiplier vector $\bar{\pi}_n$. Let $\vec{\xi}_n$ be a scenario corresponding to node n . Given primal decisions and dual multipliers for each node n , let $\mathcal{B}_n$ be an optimal basis for the following nodal LP:

$$\begin{cases} \min(c_n - \sum_{\tau=t_n+1} E[(A_{\tau t_n}^\top \bar{\pi}_\tau) | \mathcal{F}_{t_n}](\vec{\xi}_n))^\top x_n \\ A_{t_n t_n} x_n \\ x_n \end{cases} = \begin{cases} b_n - \sum_{m \in H_n: \bar{p}_m > 0} A_{t_n t_m} \bar{x}_m \\ 0 \end{cases}$$

This LP has a solution since the dual solution $\bar{\pi}_n$ obtained from the fully aggregated problem is feasible for this LP and this LP is primal feasible by construction. We can use $\bar{x}_1$, $\{\bar{\pi}_n\}_{n \in \mathcal{N}}$ and $\{\mathcal{B}_n\}_{n \in \mathcal{N}}$ to produce a *policy* for the true problem. This is accomplished by using the following equations which have their roots in sensitivity analysis of linear programming:

$$x_{2,B}(\vec{\xi}) = \mathcal{B}_2^{-1}(b_2(\vec{\xi}_2) - A_{21}\bar{x}_1) \quad (9)$$

$$x_{2,N}(\vec{\xi}) = 0 \quad (10)$$

$$x_{t_n,B}(\vec{\xi}) = \mathcal{B}_n^{-1}(b_{t_n}(\vec{\xi}_{t_n}) - \sum_{\tau=1}^{t_n-1} A_{t_n \tau} x_\tau(\vec{\xi}_\tau)) \quad (11)$$

$$x_{t_n,N}(\vec{\xi}) = 0. \quad (12)$$

This policy will be referred to as the *optimal basis prolongation* and is our estimate of the optimal solution to the true problem. For any first stage $\bar{x}_1$, and any scenario $\vec{\xi}$ the decisions $x_t(\vec{\xi})$ may be calculated recursively. Such a policy may not be feasible, i.e., there may be a set of positive measure on which the policy does not satisfy the nonnegativity constraint.

Our analysis will focus on individual nodes. Consider node n and recall that H_n is the history and F_n the future of this node. Whenever the scenario $\vec{\xi}$ is explicitly identified for node n , we let X_n denote the associated policy $\{x_{t_m}(\vec{\xi})\}_{m \in H_n}$ and explicitly recognize its dependence on the scenario. The dual solution associated with all future nodes in F_n will be denoted $\bar{\Pi}_n$. Define the salvage value function g_n , at node n , as follows:

$$\begin{cases} g_n(\vec{\xi}; X_n, \bar{\Pi}_n) := \min(c_n - \sum_{\tau=t_n+1} E[(A_{\tau t_n}^\top \bar{\pi}_\tau) | \mathcal{F}_{t_n}](\vec{\xi}))^\top x_n \\ A_{t_n t_n} x_n \\ x_n \end{cases} = \begin{cases} b(\vec{\xi}_{t_n}) - \sum_{m \in H_n: \bar{p}_m > 0} A_{t_n t_m} x(\vec{\xi}) \\ 0 \end{cases}$$

We refer to g_n as the salvage value function at node n because the cost coefficients of the above LP may be interpreted as a per unit cost that accommodates an estimated salvage value based on dual multipliers

associated with nodes following n . It is important to note that the salvage value function is also a function of the entire policy leading up to node n . Thus, $\vec{\xi}$ varies over $\bar{B}_n$, the scenarios represented by the node n . In a sense, this is a generalization of recourse value function in the two stage case, which only requires outcomes associated with the random vector for one time stage.

Upper and Lower Bounding Assumption. Let U_n denote an upper bound and L_n a lower bound for the conditional expectation $E[g_n(\vec{\xi}, X_n, \bar{\Pi}_n) | \bar{B}_n]$. We assume that whenever the salvage value function is affine over $\bar{B}_n$, the upper and lower bounds are equal to this conditional expectation.

Standard upper bounds used in stochastic programming (e.g. Edmundson-Madansky bounds) satisfy this assumption. Similarly, Jensen's lower bound for two stage problems also satisfies the assumption. We now define a gap parameter denoted δ_n :

$$\delta_n = U_n - L_n. \quad (13)$$

The random variable $\bar{\pi}_t : \Xi \rightarrow \mathbb{R}^{m_t}$ is defined as $\bar{\pi}_t(\vec{\xi}) = \bar{\pi}_m$ whenever $\vec{\xi} \in \bar{B}_m$.

Lemma 4.1. Let x_t be a random variable and $\mathcal{F}_t$ a σ -algebra for each $t \in \{1, \dots, T\}$. If $x_t \in \mathcal{F}_t$ for each $t \in \{1, \dots, T\}$ then

$$E \sum_{t=1}^{T-1} \sum_{\tau=t+1}^T E[(A_{\tau t}^\top \bar{\pi}_\tau)^\top | \mathcal{F}_t] x_t = E \sum_{t=2}^T \sum_{\tau=1}^{t-1} (A_{t\tau} x_\tau)^\top \bar{\pi}_t.$$

Proof. Since x_t is $\mathcal{F}_t$ -measurable, we can move x_t into the conditional expectation and then rearrange terms:

$$\begin{aligned} E \sum_{\tau=2}^T \sum_{t=1}^{\tau-1} E[(A_{\tau t}^\top \bar{\pi}_\tau)^\top | \mathcal{F}_t] x_t &= E \sum_{\tau=2}^T \sum_{t=1}^{\tau-1} E[(A_{\tau t} x_t)^\top \bar{\pi}_\tau | \mathcal{F}_t] \\ &= E \sum_{t=2}^T \sum_{\tau=1}^{t-1} (A_{t\tau} x_\tau)^\top \bar{\pi}_t \end{aligned}$$

□

The next theorem gives sufficient conditions for a policy to be optimal.

Theorem 4.2. Let the policy $x = (x_1, \dots, x_T)$ be an optimal basis prolongation (9-12) obtained for some tree $\mathcal{T}$. If the policy is feasible for the true problem and $\delta_n = 0$ for each node n , then the policy is optimal for the true problem.

Proof. Since $\delta_n = 0$ for each node n of the tree $\mathcal{T}$, then by linear programming sensitivity analysis applied to the salvage value problem we have for each $\vec{\xi} \in \bar{B}_n$ that

$$(c_n - \sum_{\tau=t_n+1}^T E[(A_{\tau t_n}^\top \bar{\pi}_\tau) | \mathcal{F}_{t_n}](\vec{\xi}))^\top x_{t_n}(\vec{\xi}) = (b(\vec{\xi}_{t_n}) - \sum_{m \in H_n: p_m > 0} A_{t_n t_m} x(\vec{\xi}))^\top \bar{\pi}_n.$$

Summing over all stages and applying the expectation operator then gives:

$$E[\sum_{t=1}^T (c_t - \sum_{\tau=t+1}^T E[(A_{\tau t}^\top \bar{\pi}_\tau) | \mathcal{F}_t])^\top x_t] = E[\sum_{t=1}^T (b_t - \sum_{\tau=1}^t (A_{t\tau} x_\tau)^\top \bar{\pi}_t)].$$

Since $x_t \in \mathcal{F}_t$, Lemma 4.1 implies that

$$E \sum_{t=1}^T c_t^\top x_t = E \sum_{t=1}^T b_t^\top \bar{\pi}_t.$$

This implies that the policy $x = (x_1, \dots, x_T)$ is optimal. □

We evaluate the *goodness* of our current policy for each node by evaluating the *gap parameter* δ_n as well as an *infeasibility index*. The latter will be based on the measure of the set of ξ where the policy violates the nonnegativity constraint. Define

$$\hat{\eta}_n := 1 - P(\vec{\xi} \in \Xi | x_{t_n, B}(\vec{\xi}) \geq 0). \quad (14)$$

While $\hat{\eta}_n$ is not necessarily easy to compute, an upper bound on it is often easily computed. The calculation of these probabilities is also facilitated by the fact that $x_{t_n, B}(\vec{\xi})$ is an affine function of $\vec{\xi}_{t_n}$ as can be seen from the structure of the optimal basis prolongation. We refer the reader to Prékopa [13] who discusses several methods for computing upper bounds on probability measures over polyhedral sets. In any event, let $\eta_n \geq \hat{\eta}_n$ denote such an infeasibility index for node n .

Now we are ready to present the Scenario Generation algorithm:

- **Step 0** Choose positive tolerance parameters ε_1 , ε_2 , and ε_s , and choose $\varepsilon_3 > \varepsilon_s$. Set $k := 1$.
- **Step 1** Let $\mathcal{F}^1 = \{\mathcal{F}_t\}_{t=1}^T$ be the degenerate filtration where $\mathcal{F}_t = \{\Xi^t, \emptyset\}$. Define the degenerate tree as

$$\begin{aligned} \mathcal{T}^1 &= (\mathcal{N}^1, \mathcal{A}^1) \text{ consisting of a single path,} \\ S^1 &\text{ being the single scenario, } p_S^1 = 1, \\ \bar{p}_n^1 &= 1 \quad \text{when } n \in \mathcal{N}^1, \\ q_{nm} &= 1 \quad \text{when } n \in \{1, \dots, T-1\}, m \in F_n, \\ b_n^1 &:= E[b_{t_n}]. \end{aligned}$$
- (Other initial trees are of course permissible.)
- **Step 2** (*Solve aggregated problem*) Solve the MSLP associated with the tree $\mathcal{T}^k$. This MSLP is the fully aggregated problem. The solution of this problem yields primal-dual policies $\{x_j^k, \pi_m^k : j, m \in \mathcal{N}^k\}$ and the optimal value v^k .
- **Step 3** Calculate δ_n and η_n for each node $n \in \mathcal{N}^k$ with $\bar{p}_n > \varepsilon_3$ (if a node n is such that $\bar{p}_n \leq \varepsilon_3$ then that node is ignored).
- **Step 4** (*stopping rule*) If $\delta_n < \varepsilon_1$, $\eta_n < \varepsilon_2 \quad \forall n \in \mathcal{N}^k$ and $\varepsilon_3 \leq \varepsilon_s$ then STOP; the policy generated by optimal basis prolongation is adequate for the true problem.
- **Step 5** (ε_3 reduction) If $\bar{p}_n < \varepsilon_3 \quad \forall n \in \mathcal{N}^k$ then set $\varepsilon_3 \leftarrow \varepsilon_3/2$ and go to step 3.
- **Step 6** (*splitting*) Let $L = \{n \in \mathcal{N}^k | \bar{p}_n > \varepsilon_3\}$.
 - If there is an $n \in L$ so that $\delta_n > \varepsilon_1$ then let $\bar{n}$ be such that $\delta_{\bar{n}} = \max_{n \in L} \{\delta_n\}$. Apply *tree update* procedure at node $\bar{n}$. This results in a new scenario tree $\mathcal{T}^{k+1}$. Set $k \leftarrow k+1$ and go to step 2.
 - If for all nodes $n \in L$, $\delta_n < \varepsilon_1$ but there exists a node $m \in L$ so that $\eta_m \geq \varepsilon_2$ then let $\bar{n}$ be the node in L with the largest η_n . Apply *tree update* procedure at node $\bar{n}$. This results in a new scenario tree $\mathcal{T}^{k+1}$. Set $k \leftarrow k+1$ and go to step 2.

A few remarks regarding the design of the above algorithm are in order. Using the gap parameter to choose the node to partition allows us to identify *low probability, high cost* events that may speed-up finite-time termination. With regard to node splitting based on the infeasibility index, we note that in cases where multiple random variables cause infeasibility at a node, we should consider splitting the earliest parent node leading up to the node with the high infeasibility index. Thus, consider two nodes n_1 and n_2 , such that n_2 is in the future of node n_1 . In such an instance, we recommend that node n_1 be split. An example of such a split is provided in the illustrative example presented at the end of this paper. Finally, observe that the above procedure promotes the use of parallel/distributed computing because the main computational work, which includes the calculations of δ_n and η_n , can be done node-by-node (independently). This allows the method to be parallelizable without much difficulty.

Our last result in this section is a corollary to theorem 4.2 which can be used to help terminate the algorithm in finite time.

Corollary 4.3. *Let $\mathcal{F}$ be the filtration generated by the stochastic process $(\xi_1, \dots, \xi_T)$. Let $\varepsilon > 0$ be given and let $\mathcal{T}$ be a tree, whose corresponding filtration $\hat{\mathcal{F}}$ is a subfiltration of $\mathcal{F}$, with the property that for each node n either (i) $\delta_n = 0$ and $\eta_n = 0$ or (ii) $\bar{p}_n < \varepsilon$. Then there exists a tree $\mathcal{T}_\varepsilon$ so that (i) each node m of $\mathcal{T}_\varepsilon$ has $\bar{p}_m < \varepsilon$ and (ii) an optimal policy for MSLP over tree $\mathcal{T}$ when prolonged to $\mathcal{T}_\varepsilon$ is also optimal for MSLP with tree $\mathcal{T}_\varepsilon$.*

Proof. The tree $\mathcal{T}_\varepsilon$ may not be unique. One way of producing it is the following algorithm: (i) split the node n of $\mathcal{T}$ with the largest $\bar{p}_n$ which is greater than ε , (ii) apply the tree-update procedure, and (iii) if

this new tree has a node n with $\bar{p}_n > \varepsilon$ start over with (i) for this new tree, otherwise the new tree is $\mathcal{T}_\varepsilon$. Now applying theorem 4.2 completes the proof. $\square$

It is appropriate to compare our approach to other successive refinement approaches for MSLP. Frauendorfer's approach [6], which is a generalization of his barycentric method for two-stage problems, uses a simplex in the space of scenarios to approximate the given scenario tree. The lower bounds are obtained using the Jensen bound, whereas, upper bounds are consistent with the barycentric method. Each extreme point of such a simplex is an "extreme" scenario, and upper bounding requires the solution of as many scenario LPs as there are extreme points of the simplex. Thus, if each stage has N random variables, the simplex for a T period problem has $(N + 1)^T$ extreme points. As a result, upper bounding in this method may require the solution of a fairly large number of scenario LPs. However, this method is provably convergent. In contrast, the method proposed by Edirisinghe [4] avoids the calculation of upper bounds, and instead, works on improving the lower bounds using first and second moments of the random variables. This approach requires us to view the MSLP as a two-stage SLP in which non-anticipativity constraints are imposed algebraically on decisions beyond the first stage. However, including the entire collection of non-anticipativity constraints can be computationally burdensome. To alleviate this difficulty, Edirisinghe uses an aggregated set of non-anticipativity constraints. The multipliers used for the aggregation correspond to a certain probability measure derived from the probability of a scenario (assuming there are finitely many) together with probability of an "extreme" scenario (corresponding to an extreme point of a simplex containing all scenarios in the tree). This approach may be viewed as a heuristic, since no asymptotic results are known at this time. Moreover, this work relies on having a finite number of scenarios in the problem definition. Nevertheless, computations provided in Edirisinghe [4] do support the idea that problems of practical size may be addressed by this approach.

5. Convergence. In this section we discuss convergence of the SG algorithm. The algorithm produces a sequence of fully aggregated problems $\{P(\mathcal{F}^k, \mathcal{F}^k)\}_{k=1}^\infty$ where k is the iteration count of the algorithm and $\mathcal{F}^k$ is the filtration corresponding to the tree $\mathcal{T}^k$. The results below show that under certain conditions, as $k \rightarrow \infty$, $v(P(\mathcal{F}^k, \mathcal{F}^k)) \rightarrow v(P(\mathcal{F}, \mathcal{F}))$.

Since we will be relying upon Olsen [11, theorem 2.1] we need to verify the conditions of that theorem. To this end we begin by introducing step function prolongations. Let x_n be the solution at node n to the fully aggregated problem $P(\hat{\mathcal{F}}, \hat{\mathcal{F}})$. The step function prolongation $px_n : \bar{B}_n \rightarrow \mathbb{R}^{n_t}$ is given by

$$px_n(\cdot) = x_n.$$

Let the policy formed by all these prolongations be denoted px .

The restriction operator r^k restricts a function u with domain Ξ to the current scenario tree $\mathcal{T}^k$. This has the effect of making the restricted function measurable with respect to the filtration associated with the scenario tree. One obvious choice for this operator is conditional expectation, i.e. given a function u with domain Ξ , $r^k u(\vec{\xi}) = E[u | \mathcal{F}_t](\vec{\xi})$.

The following conditions are sufficient to satisfy the hypotheses of Olsen [11, theorem 2.1]:

- (a) the feasible set of $P(\mathcal{F}, \mathcal{F})$ (the true MSLP) is nonempty and bounded;
- (b) $P(\mathcal{F}, \mathcal{F})$ has an optimal solution u which is continuous;
- (c) $r^k u$ is feasible for each aggregated MSLP $P(\mathcal{F}^k, \mathcal{F}^k)$;
- (d) $\|pr^k u - u\|_2 \rightarrow 0$ as $k \rightarrow \infty$; and
- (e) $\|pr^k b - b\|_2 \rightarrow 0$ and $\|pr^k c - c\|_2 \rightarrow 0$ as $k \rightarrow \infty$.

Since we assume that b_t is an affine function for $t \in \{1, \dots, T\}$, every feasible solution of $P(\mathcal{F}, \mathcal{F})$ is also continuous. Thus in particular if u is an optimal solution of $P(\mathcal{F}, \mathcal{F})$ then u is continuous and condition (b) would be satisfied.

We first prove convergence when all random variables have finite support and then proceed to the continuous case.

Proposition 5.1. *Assume for each $t \in \{1, \dots, T\}$ that ξ_t has finite support. Set $\varepsilon_1 = 0$, $\varepsilon_2 = 0$ and set ε_s be set to a value smaller than the smallest $\bar{p}_n$. Then (i) the SG algorithm terminates in a finite*

number of steps and (ii) when it terminates the optimal value v^k of the approximating problem equals the optimal value of the true MSLP and the optimal solution x^k is an optimal solution of the true MSLP.

Proof. At each iteration the algorithm either stops because the optimality conditions of Theorem 4.2 are satisfied or it refines the aggregation by splitting some node of the current scenario tree. Hence in the worst case the algorithm reproduces the true tree and since there are only a finite number of scenarios, it does this in a finite number of steps. When the algorithm terminates we have $\delta_n = 0$ for all n and the measure of infeasibility is also 0. Therefore we have (i) and (ii) above. $\square$

When the random elements of the problem are continuous we no longer have a guarantee that the algorithm terminates in a finite number of steps. In this case we show asymptotic consistency.

Theorem 5.2. Set $\varepsilon_1 = 0$, $\varepsilon_2 = 0$ and $\varepsilon_s = 0$. If the MSLP $P(\mathcal{F}, \mathcal{F})$ (a) has an optimal solution and (b) has a bounded feasible set then every weak limit point of the sequence of step function prolongations $\{(px)^k\}_{k=1}^\infty$ is an optimal solution of the true problem. This conclusion implies that as $k \rightarrow \infty$ we have

$$v(P(\mathcal{F}^k, \mathcal{F}^k)) \rightarrow v(P(\mathcal{F}, \mathcal{F})).$$

Proof. Whenever the algorithm terminates in finitely many steps, we have $\delta_n = \eta_n = 0$ for all n . Hence by theorem 4.2 we have an optimal solution and optimal value for the true problem as well as a discrete representation (the scenario tree) of the continuous stochastic process. On the other hand, consider the case when the algorithm does not terminate in a finite number of steps. Step 5 of the algorithm reduces ε_3 by halving this parameter at each major iteration. Thus the algorithm produces a sequence of scenario trees $\mathcal{T}^k$ whose corresponding probability measures μ^k weakly converge to the true measure P . Since $c = (c_1, \dots, c_T)$ is constant, b_t is affine and there is a continuous solution u of the MSLP, conditions (b), (d) and (e) above are satisfied. Using the restriction operator as defined above on u , condition (c) is satisfied. Finally, since we assume the MSLP has a bounded, nonempty feasible set condition (a) is also met. The conclusion of Olsen [11, theorem 2.1] finishes the proof. $\square$

6. Illustration of the Scenario Generation Algorithm. We illustrate the algorithm with the following 3-stage problem provided by S. Siegrist (University of Zürich). We provide significant details for Iteration 1, and then, summarize the remaining calculations in the interest of brevity.

$$\begin{aligned} \min \quad & 5x_1 + E[12y_1(\xi_2) + 12y_2(\xi_2)] + E[10z_1(\xi_2, \xi_3) + 10z_2(\xi_2, \xi_3)] \\ \text{subject to} \quad & x_1 + x_2 = 1 \\ & x_1 - x_2 + y_1(\xi_2) - y_2(\xi_2) = \xi_2 \\ & y_1(\xi_2) - y_2(\xi_2) + z_1(\xi_2, \xi_3) - z_2(\xi_2, \xi_3) = \xi_3 \\ & x_1, x_2, y_1, y_2, z_1, z_2 \geq 0. \end{aligned}$$

In this problem ξ_2 and ξ_3 are independent and both are uniform random variables - the first over $[0, 4]$, the second over $[4.2, 5.2]$.

In what follows we give results of calculations as well as depictions of scenario trees in Figures 1-5. The nodes are numbered left-to-right, top-to-bottom and renumbered for each tree. In addition, the probability of each node is given along with the set of scenarios it represents.

Initialization

We initialize the process with the tree in Figure 1. This tree is obtained by replacing the random variables with their expectations.

Iteration 1

We solve the aggregated LP:

$$\min 5x_1 + 12y_1 + 12y_2 + 10z_1 + 10z_2$$

$$\begin{aligned}
\text{subject to } x_1 + x_2 &= 1 \\
x_1 - x_2 + y_1 - y_2 &= 2 \\
y_1 - y_2 + z_1 - z_2 &= 4.7 \\
x_1, x_2, y_1, y_2, z_1, z_2 &\geq 0.
\end{aligned}$$

- Value of LP = 53
- Primal solution: $x_1^1 = 0, x_2^1 = 1, y_1^1 = 3, z_1^1 = 1.7$, all others = 0
- Dual solution: $u_1 = 2, u_2 = 2, u_3 = 10$ (Note: the subscripts correspond to node numbers (constraints) on the tree. These values have not been scaled using nodal probabilities.)
- We split node 2 (see details for this below).

Solve Nodal Relaxations.

- *Node 3*

$$\begin{aligned}
\min \quad & 10z_1 + 10z_2 \\
\text{subject to } & z_1 - z_2 = 4.7 - (y_1 - y_2) = 1.7 \\
& z_1, z_2 \geq 0.
\end{aligned}$$

- $\mathcal{B}_3 = 1$

- *Node 2*

$$\begin{aligned}
\min \quad & 2y_1 + 22y_2 \\
\text{subject to } & y_1 - y_2 = 2 - (x_1 - x_2) = 3 \\
& y_1, y_2 \geq 0.
\end{aligned}$$

- $\mathcal{B}_2 = 1$

Tree Refinement:

- *Policy formulation at node 2*

$$(y_1, y_2) = (\xi_2 + 1, 0)$$

Since $\xi_2 \geq 0$ it follows that this policy is feasible for node 2.

- *Policy formulation at node 3*

$$(z_1, z_2) = (\xi_3 - \xi_2 - 1, 0)$$

Since the minimum value of $\xi_3 - \xi_2 - 1$ is negative over the set $[0, 4] \times [4.2, 5.2]$, this policy is infeasible for this node. We will choose to split the node with the lowest index in the above inequality. So we split node 2.

The resulting tree is depicted in Figure 4.

Iteration 2

We solve the aggregated LP:

$$\begin{aligned}
\min \quad & 5x_1 + 9.6y_{11} + 9.6y_{21} + 2.4y_{12} + 2.4y_{22} + 8z_{11} + 8z_{21} + 2z_{12} + 2z_{22} \\
\text{subject to } & x_1 + x_2 = 1 \\
& x_1 - x_2 + y_{11} - y_{21} = 1.6 \\
& x_1 - x_2 + y_{12} - y_{22} = 3.6 \\
& y_{11} - y_{21} + z_{11} - z_{21} = 4.7 \\
& y_{12} - y_{22} + z_{12} - z_{22} = 4.7 \\
& x_1, x_2, y_{11}, y_{21}, y_{12}, y_{22}, z_{11}, z_{21}, z_{12}, z_{22} \geq 0.
\end{aligned}$$

- Value of LP = 53, Primal solution: $x_1^2 = 0, x_2^2 = 1, y_{11}^2 = 2.6, y_{12}^2 = 4.6, z_{11}^2 = 2.1, z_{12}^2 = 0.1$, all others = 0, Dual solution: $u_1 = 2, u_2 = 1.6, u_3 = 0.4, u_4 = 8, u_5 = 2$
- We split node 5 (see Figure 5 for the resulting tree)

Iteration 3

We solve the aggregated LP:

$$\begin{aligned}
 \min \quad & 5x_1 + 9.6y_{11} + 9.6y_{21} + 2.4y_{12} + 2.4y_{22} + 8z_{11} + 8z_{21} \\
 & + 1.6z_{12} + 1.6z_{22} + 0.4z_{13} + 0.4z_{23} \\
 \text{subject to} \quad & x_1 + x_2 = 1 \\
 & x_1 - x_2 + y_{11} - y_{21} = 1.6 \\
 & x_1 - x_2 + y_{12} - y_{22} = 3.6 \\
 & y_{11} - y_{21} + z_{11} - z_{21} = 4.7 \\
 & y_{12} - y_{22} + z_{12} - z_{22} = 4.6 \\
 & y_{12} - y_{22} + z_{13} - z_{23} = 5.1 \\
 & x_1, x_2, y_{11}, y_{21}, y_{12}, y_{22}, z_{11}, z_{21}, z_{12}, z_{22}, \\
 & z_{13}, z_{23} \geq 0.
 \end{aligned}$$

- Value of LP = 53, Primal solution: $x_1^3 = 0, x_2^3 = 1, y_{11}^3 = 2.6, y_{12}^3 = 4.6, z_{11}^3 = 2.1, z_{13}^3 = 0.5$, all others = 0, Dual solution: $u_1 = 2.5, u_2 = 1.6, u_3 = 0.9, u_4 = 8, u_5 = 1.1, u_6 = 4$
- We split node 5 (see Figure 6 for resulting tree)

Iteration 4

We solve the aggregated LP:

$$\begin{aligned}
 \min \quad & 5x_1 + 9.6y_{11} + 9.6y_{21} + 2.4y_{12} + 2.4y_{22} + 8z_{11} + 8z_{21} \\
 & + .8z_{12} + .8z_{22} + .8z_{13} + .8z_{23} + .4z_{14} + .4z_{24} \\
 \text{subject to} \quad & x_1 + x_2 = 1 \\
 & x_1 - x_2 + y_{11} - y_{21} = 1.6 \\
 & x_1 - x_2 + y_{12} - y_{22} = 3.6 \\
 & y_{11} - y_{21} + z_{11} - z_{21} = 4.7 \\
 & y_{12} - y_{22} + z_{12} - z_{22} = 4.4 \\
 & y_{12} - y_{22} + z_{13} - z_{23} = 4.8 \\
 & y_{12} - y_{22} + z_{14} - z_{24} = 5.1 \\
 & x_1, x_2, y_{11}, y_{21}, y_{12}, y_{22}, z_{11}, z_{21}, z_{12}, z_{22}, \\
 & z_{13}, z_{23}, z_{14}, z_{24} \geq 0.
 \end{aligned}$$

- Value of LP = 53.1, Primal solution: $x_1^4 = 0.1, x_2^4 = .9, y_{11}^4 = 2.4, y_{12}^4 = 4.4, z_{11}^4 = 2.3, z_{13}^4 = 0.4, z_{14}^4 = 0.7$, all others = 0, Dual solution: $u_1 = 2.5, u_2 = 1.6, u_3 = 0.9, u_4 = 8, u_5 = .3, u_6 = .8, u_7 = .4$
- We split node 3 (see Figure 7)

Iteration 5

We solve the aggregated LP:

$$\begin{aligned}
 \min \quad & 5x_1 + 9.6y_{11} + 9.6y_{21} + 1.2y_{12} + 1.2y_{22} + 1.2y_{13} + 1.2y_{23} + 8z_{11} + 8z_{21} \\
 & + .4z_{12} + .4z_{22} + .4z_{13} + .4z_{23} + .2z_{14} + .2z_{24} + .4z_{15} + .4z_{25} + .4z_{16} \\
 & + .4z_{26} + .2z_{17} + .2z_{27} \\
 \text{subject to} \quad & x_1 + x_2 = 1 \\
 & x_1 - x_2 + y_{11} - y_{21} = 1.6 \\
 & x_1 - x_2 + y_{12} - y_{22} = 3.4 \\
 & x_1 - x_2 + y_{13} - y_{23} = 3.8 \\
 & y_{11} - y_{21} + z_{11} - z_{21} = 4.7 \\
 & y_{12} - y_{22} + z_{12} - z_{22} = 4.4 \\
 & y_{12} - y_{22} + z_{13} - z_{23} = 4.8
 \end{aligned}$$

$$\begin{aligned}
y_{12} - y_{22} + z_{14} - z_{24} &= 5.1 \\
y_{13} - y_{23} + z_{15} - z_{25} &= 4.4 \\
y_{13} - y_{23} + z_{16} - z_{26} &= 4.8 \\
y_{13} - y_{23} + z_{17} - z_{27} &= 5.1 \\
x_1, x_2, y_{11}, y_{21}, y_{12}, y_{22}, y_{13}, y_{23}, \\
z_{11}, z_{21}, z_{12}, z_{22}, z_{13}, z_{23}, z_{14}, \\
z_{24}, z_{15}, z_{25}, z_{16}, z_{26}, z_{17}, z_{27} &\geq 0.
\end{aligned}$$

- Value of LP = 53.2, Primal solution: $x_1^5 = 0.2, x_2^5 = .8, y_{11}^5 = 2.2, y_{12}^5 = 4.0, y_{13}^5 = 4.4, z_{11}^5 = 2.5, z_{12}^5 = 0.4, z_{13}^5 = 0.8, z_{14}^5 = 1.1, z_{16}^5 = 0.4, z_{17}^5 = 0.7$, all others = 0, Dual solution: $u_1 = 2.5, u_2 = 1.6, u_3 = 0.2, u_4 = 0.7, u_5 = 8, u_6 = .4, u_7 = .4, u_8 = .2, u_9 = -0.1, u_{10} = .4, u_{11} = .2$
- Using arguments similar to Corollary 4.3, we can show that this solution is optimal.

7. Conclusions and Future Research. Stochastic programming models of practical applications lead to some of the largest optimization problems. The size of these problems is linked directly to the number of scenarios used to model uncertainty. It is therefore customary to solve an approximation generated by either an aggregation or discretization of the probability model representing uncertainty. For this reason, a discrete scenario tree is a prerequisite for traditional stochastic programming (Birge and Louveaux [1]). Even in instances where the original problem is described by a discrete scenario tree, the number of scenarios may be so large, that traditional algorithms require some aggregation (of scenarios) so that the optimization problem can be solved in reasonable time. However, standard stochastic programming approaches have not yet provided any prescription as to how such aggregated scenario trees may be generated. In this paper, we have described an algorithm that generates a sequence of scenario trees which not only provide asymptotic convergence, but also provide a measure (using $\sum_n p_n \delta_n$) of optimality of the first stage decision. Moreover, the algorithm provides a sequence of policies (or prolongations) which can be used to adapt to actual realizations, provided the policy is feasible with respect to the realization. Finally, by specifying the feasibility tolerance (ε_2), the user may control the likelihood that the policy generated by the algorithm is feasible in any stage of the decision process. It is interesting to note that this feature unites two of the main approaches in stochastic programming, namely, recourse problems and probabilistically constrained problems.

The approach presented here merits further investigation in several directions. We begin by discussing changes that may potentially improve the performance of the current algorithm, and then discuss extensions to allow more general classes of problems. First, we should mention that the estimates of infeasibility used within the algorithm can be improved by using Prékopa's [13] approaches of bounding probability measures over polyhedral sets. These estimates may not only help terminate the algorithm faster, but also generate scenario trees that are smaller. Another feature that one may consider including in the algorithm is the notion of "aggregation", by which we mean that the algorithm should allow for the possibility of combining certain nodes, without altering the objective value associated with the (new) aggregated tree. Both these changes will allow us to control the growth of the scenario tree.

In discussing extensions to the class of problems addressed by our method, we should first recognize that the current version relies on the convexity of LP value functions. Therefore the current algorithm is restricted to only allow randomness in the right-hand side vectors. Extensions that allow random objective coefficients, and constraint matrices are certainly worth investigating. Another avenue worth pursuing arises from the need to ease calculations associated with upper bounds used in the method. It is well known that when the number of random variables grows, upper bounds become more cumbersome to calculate, and sampling schemes become more attractive. An extension that incorporates sampling within the algorithm of this paper is therefore worth considering. We believe that these investigations will provide the basis for generating both scenarios and decisions simultaneously.

Acknowledgments. This research has been partially supported by grants DDM-9978780 and CISE-9975050 from the National Science Foundation. The second author is grateful to Peter Kall and Janos Mayer for many discussions concerning an earlier version of the algorithm (without optimal basis prolongations and feasibility indices). Those discussions helped crystalize the need for the current version. Both authors thank Simon Siegrist for reading and providing a counterexample for an earlier version of this paper. Both authors also thank the anonymous referees for helping improve the exposition.

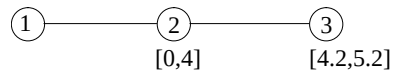


Figure 3: Initial scenario tree $\mathcal{T}^0$.

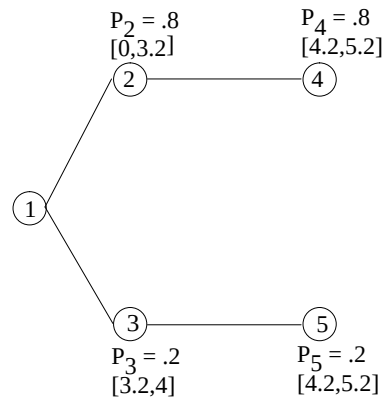


Figure 4: Iteration 1 - scenario tree $\mathcal{T}^1$.

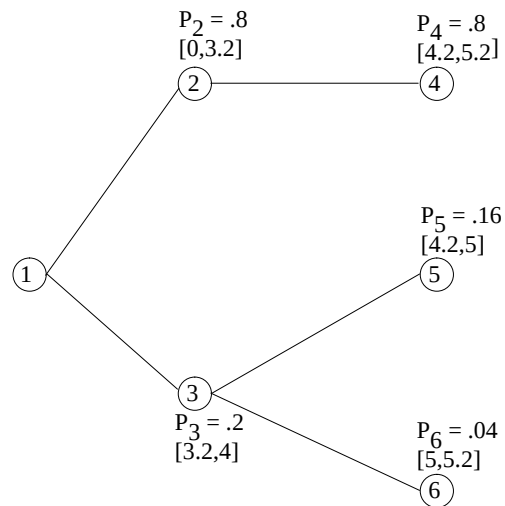
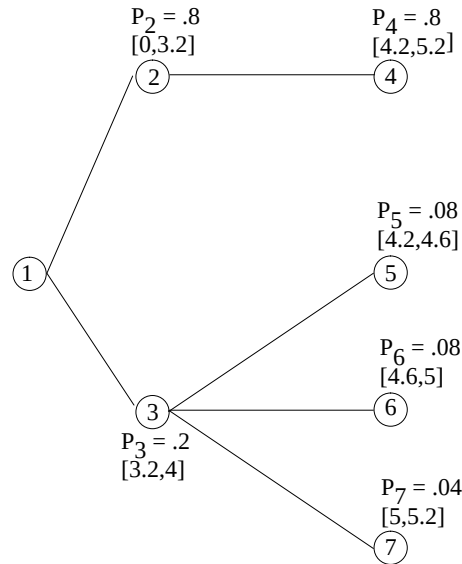
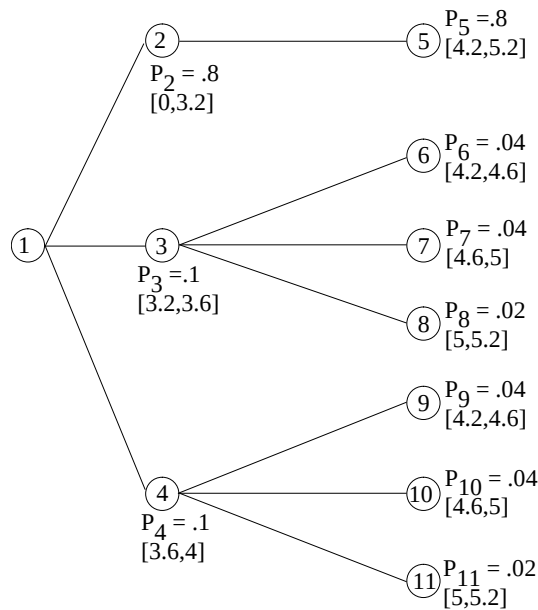


Figure 5: Iteration 2 - scenario tree $\mathcal{T}^2$.

Figure 6: Iteration 3 - scenario tree T^3 .Figure 7: Iteration 4 - scenario tree T^4 .

References

- [1] J.R. Birge and F.V. Louveaux. *Introduction to Stochastic Programming*. Springer Series in Operations Research. Springer-Verlag, 1997.
- [2] D.R. Carino, D.H. Myers, and W.T. Ziemba. Concepts, technical issues and uses of the Russell-Yasuda-Kasai financial planning model. *Operations Research*, 46(4):450–462, 1998.
- [3] R. Durrett. *Probability: Theory and Examples*. Duxbury Press, Belmont, California, 2nd edition, 1995.
- [4] N. Edirisinghe. Bounds-based approximations in multistage stochastic programming. *Annals of Operations Research*, 85:103–127, 1999.
- [5] N. Edirisinghe and W.T. Ziemba. Implementing bounds-based approximations in convex-concave two-stage stochastic programming. *Mathematical Programming*, 78(2):314–340, 1996.
- [6] K. Frauendorfer. Barycentric scenario trees in convex multistage stochastic programming. *Mathematical Programming B*, 75:277–293, 1996.
- [7] K. Frauendorfer and P. Kall. A solution method for slp recourse problems with arbitrary multivariate distributions - the independent case. *Problems of Control and Information Theory*, 17:177–205, 1988.

- [8] K. Hoyland and S.W. Wallace. Generating scenario trees for multi-stage decision problems. *Management Science*, 47(2):295–307, 2001.
- [9] N. Growe-Kuska J. Dupacova and W. Romisch. Scenario reduction in stochastic programming: an approach using probability metrics. *preprint*, 2000.
- [10] M.P. Nowak and W. Romisch. Stochastic lagrangian relaxtion applied to power scheduling in a hydro-thermal system under uncertainty. *Annals of Operations Research*, 100:251–272, 2000.
- [11] P. Olsen. Discretizations of multistage stochastic programming. *Mathematical Programming Study*, 6:111–124, 1976.
- [12] G. C. Pflug. Scenario tree generation for multiperiod financial optimization by optimal discretization. *Mathematical Programming*, 89:251–271, 2000.
- [13] A. Prékopa. *Stochastic Programming*. Kluwer Academic Publishers, Dordrecht, The Netherlands, 1995.
- [14] K. Boskma R.J. Peters and H.A.E. Kuper. Stochastic programming in production planning: a case with non-simple recourse. *Statistica Neerlandica*, 31:113–126, 1977.
- [15] R. T. Rockafeller. Duality and optimality in multistage stochastic programming. *Annal of Operations Research*, 85:1–19, 1999.
- [16] S. E. Wright. Primal-dual aggregation and disaggregation for stochastic linear programs. *Mathematics of Operations research*, 19(4):893–908, November 1994.